Toward Skill Evolution for Blind Vulnerability Analysis

Authors omitted in working draft[a]

[a] *Affiliations omitted in working draft*

Abstract

Reusable natural-language skills are attractive for blind vulnerability-analysis agents because they can encode recurring procedures: identify a dispatcher, narrow to a branch, trace attacker-controlled data, and localize a sink. However, a static skill library is not enough. Across firmware families and bug patterns, a skill that helps one target may be redundant, ineffective, or even harmful on another. The core problem is therefore not only how to write skills, but how to evolve them from completed runs without polluting the live library seen by later analyses. We study this problem as *evidence-constrained skill evolution* for blind vulnerability analysis. Our setting couples blind remote analysis with post-run external confirmation: an agent solves a task on an isolated virtual machine, the local system scores the final answer and checks it against source predicates, logic harnesses, behavioral oracles, or sanitizer-backed evidence, and only then may an evolution service propose and validate candidate skill updates. We present a split-plane architecture that separates analysis from confirmation, a run archival scheme that links selected skills to immutable session segments, a lightweight run-level feedback interface for candidate generation, and a publication gate that stages candidate skills before they can enter the live library. Empirically, we organize the evidence as a hierarchy rather than a single benchmark number. A leakage-controlled 84-run three-condition rerun shows that earlier near-perfect oracle performance was inflated by answer-bearing skill content; after cleanup, oracle-skill still outperforms weak-skill and no-skill while sharply reducing decoy hits. A larger frozen 153-run necessity study across 17 firmware cases confirms the effect under a stricter protocol: oracle-skill achieves 46.7% true hits versus 11.1% for weak-skill and 6.7% for no-skill, while reducing decoy hits from 28/45 and 24/45 to 2/45. Complementary studies show that skill specificity is family-dependent: a distinguishing skill raises Belkin F9K1122-

family accuracy from 20.0% to 66.7%, but the same design principle degrades Tenda FH451-family accuracy from 40.0% to 26.7%. Across 94 archived gate decisions, candidate generation is not the main bottleneck; defensible publication into the live library is. Replay-based gates can reject clearly poor candidates, but they still struggle to separate genuine improvement from harmless redundancy when the baseline model already performs well. The resulting contribution is a discovery-oriented research scaffold for studying when validated vulnerability-analysis runs should, and should not, become reusable skills.

Keywords: LLM agent, skill evolution, vulnerability analysis, external validation, blind benchmark, evidence-grounded feedback

---

1. Introduction

LLM agents increasingly use external memories, reusable instructions, and natural-language skills to improve over repeated tasks. Systems such as Voyager [1] and ExpeL [2] show that agents can accumulate procedural knowledge from interaction and reuse it later. SkillClaw [3] extends this direction to a shared skill ecosystem, while recent work studies context-to-skill induction ([4]), on-policy skill distillation ([5]), environment-grounded skill formation ([6, 7]), and verifier-guided skill refinement ([8, 9]).

This line of work is especially relevant to vulnerability analysis. Blind analysis and discovery-oriented auditing repeatedly require the same kinds of procedural moves: locate a dispatcher, identify a reachable branch, trace attacker-controlled inputs, follow dataflow toward a sink, and decide whether the path is exploit-relevant. Without reusable skills, an agent must re-learn these procedures on each target, and it often drifts toward a salient but wrong handler, sink, or vulnerability family.

At the same time, a static skill library is not enough. Our own firmware experiments, together with recent work on harmful skills and self-improvement failures [10, 11], suggest that a skill that helps one case may be ineffective or harmful on another. Security-specific work has likewise started to evolve reusable playbooks for auditing [12] and to construct realistic long-horizon benchmarks [13]. The natural next question is therefore not only why agents need skills, but why those skills must evolve from real runs rather than remain static templates.

That requirement immediately creates a harder problem. In a real

vulnerability-analysis workflow, the difficult question is not whether an LLM or agent can produce a candidate skill revision. The difficult question is whether a completed run contains reusable procedural knowledge that deserves to enter the *live* skill library seen by later runs. This question is more stringent than ordinary task success. A run can be correct yet redundant, correct yet overfit to a single instance, or correct yet unsafe to generalize. The engineering surface therefore includes not only skill retrieval and revision, but also run archival, evidence collection, candidate staging, and admission control.

Vulnerability analysis is a particularly instructive domain for studying this problem because it provides something many generic agent tasks do not: *external, executable confirmation*. Depending on the case, a claim can be checked by source predicates, logic harnesses, behavioral oracles, or sanitizer-backed crashes. These signals do not prove causal skill contribution, but they do let us constrain the evolution loop using evidence that is independent of the model's own narrative. This suggests a different research focus from prior work on skill generation or pure retrieval: rather than asking only how to produce better skills, we can ask how to govern the transition from a *validated run* to a *live reusable skill* in a blind vulnerability-analysis workflow.

Our goal is therefore narrower and more operational than a general theory of skill learning. We do not claim a new retrieval algorithm, a new revision model, or a complete solution to causal attribution. Instead, we study a full vulnerability-analysis loop in which a blind run executes on a remote analysis environment, selected skills are injected server-side, the final answer is scored and externally confirmed on a separate local plane, the resulting record is transformed into a candidate update, and the update is published only if it passes an explicit gate. This framing lets us ask three concrete questions: how unstable skill effects are in blind security tasks; whether lightweight post-run feedback is useful enough to drive candidate generation; and whether publication control, rather than candidate generation, is the real bottleneck in practice.

This paper makes four contributions:

1. **A skill-evolution framework for blind vulnerability analysis.** We formulate the transition from a completed blind run to a reusable live skill as a structured lifecycle involving execution, confirmation, archival, candidate generation, and publication control.

2. **A split-plane architecture for blind analysis and local confir-**

mation. We design a remote-analysis / local-confirmation separation that preserves blind execution while still supporting post-run evidence processing, session archival, and skill evolution.

3. **An evidence-grounded feedback and publication pipeline.** We show how externally scored runs can be converted into candidate updates while preserving target-identity information, decoy-aware diagnostics, and validation-gated publication into the live library.

4. **A staged empirical study of why skills help, fail, and remain hard to publish.** Through clean reruns, frozen necessity studies, family-specific distinguishing experiments, and archived gate analysis, we show that skill usage can materially alter vulnerability localization quality, but the effect is heterogeneous, family-dependent, and not yet sufficient to prove strict necessity in all cases.

The intended scope of this paper is therefore a *framework paper with initial empirical evidence*. Our objective is to establish the research problem, the system structure, and the first round of measurable findings. We do not claim that autonomous skill improvement has already been solved. Instead, we argue that the field needs a cleaner way to study when validated security runs should, and should not, become persistent skills, and we present a first operational scaffold for that study.

The remainder of the paper is organized as follows. Section 2 formalizes the skill-evolution problem and its two key subproblems: run-level attribution and safe publication. Section 3 presents the framework. Section 4 describes the prototype. Section 5 reports experimental methodology and results. Section 7 discusses implications. Section 8 surveys related work. Section 9 addresses threats to validity. Section 10 concludes.

2. Problem Formulation

2.1. Task Run and Validation Vector

Let a task run be

$$r = (x, S_r, \tau, y, E), \tag{1}$$

where x is a blind vulnerability-analysis task, S_r is the set of skills selected and injected for the run, τ is the agent trajectory, y is the final answer

(predicted files, functions, root cause, and evidence), and E is externally collected evidence.

We retain a validation vector rather than a single success label:

$$V(r) = (q_{\text{task}},\ q_{\text{confirm}},\ a_{r,s})\,. \tag{2}$$

Here q_{task} measures whether the answer localizes and explains the ground-truth vulnerability; q_{confirm} represents the strength of external source or runtime evidence; and $a_{r,s}$ records *observed relevance* between skill s and the task.

We deliberately call $a_{r,s}$ *observed relevance* rather than *causal contribution*. This distinction is the central problem of this paper.

2.2. The Skill Attribution Problem

Suppose a run achieves high q_{task} and high q_{confirm}: the agent correctly identified the vulnerability file, function, and root cause, and the result was confirmed by a sanitizer crash or source predicate. Can we conclude that the injected skill s causally contributed to this success?

In general, no. Consider the following decomposition. Let y_0 be the answer the model would produce *without* skill s (the counterfactual baseline), and let y_s be the answer *with* skill s. The causal effect of s on the task outcome is:

$$\Delta(s) = q_{\text{task}}(y_s) - q_{\text{task}}(y_0). \tag{3}$$

If $\Delta(s) > 0$, the skill helped. If $\Delta(s) \leq 0$, it did not help or was harmful. However, computing $\Delta(s)$ requires running the agent twice—once with s and once without—which doubles the computational cost per attribution. More importantly, LLM outputs are stochastic: a single pair of runs may not reflect the true expected effect, requiring multiple repeated trials for statistical power.

Prior work addresses this through counterfactual ablation. Differential comparison between a skill-guided run and a no-skill reference run [10], and Shapley-style step valuation over skill traces [14, 15], both provide stronger attribution signals than single-run observation. However, Shapley-style estimation requires evaluating many subsets of skill steps, making it expensive for long skills.

The key question we investigate is: *can executable domain evidence provide a cheaper approximation of* $\Delta(s)$ *without running counterfactual ablations?*

2.3. The Safe Skill Publication Problem

When a run is completed and validated, the system may generate a candidate skill revision Δs derived from the run's trajectory and feedback. The question is whether to publish Δs into the live skill library that subsequent agents will use.

Following the principle that "evolution optimizes task outcomes rather than procedure safety" [11], we require a publication gate:

$$\text{Publish}(\Delta s) = \mathbb{I}\left[\text{EvidenceSufficient}(\Delta s) \ \wedge \ \text{FollowupPass}(\Delta s)\right]. \quad (4)$$

Here EvidenceSufficient checks whether the originating run had adequate evidence to justify a skill update, and FollowupPass runs a follow-up validation with the candidate skill to check whether it maintains or improves performance.

The structural challenge is in FollowupPass. In a *replay-only* paradigm, the follow-up runs the candidate skill on the same or similar task and checks whether the result passes validation. But this only measures $q_{\text{task}}(y_{\Delta s})$, not $\Delta(\Delta s) = q_{\text{task}}(y_{\Delta s}) - q_{\text{task}}(y_0)$. Without the counterfactual baseline y_0, the gate cannot distinguish "the skill helped" from "the model would have succeeded anyway." When the gate requires *strict improvement* (mean score with candidate $>$ mean score without candidate), this leads to systematic rejection of candidates that are not harmful but merely redundant—a high false-negative rate.

2.4. Research Hypotheses

The study is guided by three hypotheses:

H1: Clean skill effect. Under leakage-controlled blind execution, stronger domain skills should increase true-hit rate and suppress decoy-hit rate relative to weak-skill and no-skill controls, but the effect remains case-dependent.

H2: Specificity boundary. More specific family-level guidance can recover part of the residual error left by generic skills, but only for families that expose stable discriminative cues.

H3: Gate limitation. A replay-only publication gate suffers a structural false-negative limitation that cannot be resolved by parameter tuning alone.

3. Evidence-Constrained Skill Evolution Framework

We propose a framework that connects blind task execution, skill injection auditing, external evidence confirmation, evidence-conditioned feedback, and gated candidate publication. The framework is designed around a simple principle: in vulnerability analysis, post-run evidence should constrain both what we infer about a skill and what we allow the system to retain for future use. The goal is not to make the evolution service omniscient. The goal is to ensure that the path from a completed run to a persistent live skill is explicit, inspectable, and conservative enough for security work.

3.1. Leakage-Controlled Blind Execution

Each evaluation begins from a benchmark specification containing private ground truth and a separately rendered blind prompt. The agent receives the target workspace and the analysis objective, but not the answer fields, confirmation scripts, or solution-bearing artifact names. The benchmark controller retains the expected file, function, root cause, and confirmation policy.

The analysis runs on a remote virtual machine, separated from the benchmark ground truth and from the local service that evaluates the result. This separation is methodologically important: it prevents inadvertent answer leakage through shared filesystems or local environment artifacts, and it mirrors real-world deployment where the analyst's environment is isolated from the vulnerability database.

3.2. Auditable Skill Injection

For each run, the system records the selected skill names, the injected prompt content hash, the request session identifier, and the immutable session segment consumed by the evolution process.

This recording addresses a prerequisite for studying skill evolution. Any claim about a skill's effect is uninterpretable if the intended skill cannot be linked to a specific request segment, or if the injected content cannot be reconstructed after the run. Injection auditing therefore does not solve attribution, but it narrows the question from "was the skill present?" to the more meaningful "what procedural prior was exposed to the model in this specific run, and how should that exposure be archived for later judgment?"

3.3. Multi-Evidence External Confirmation

The agent's output is evaluated along two complementary dimensions. First, *structured scoring* checks the predicted vulnerability identity, file, function, root cause, and cited evidence against ground truth. Second, *case validators* test claims against the target environment.

We classify evidence into four categories, each making a different claim about the vulnerability:

- **Source-backed evidence:** verifies a concrete code predicate or boundary relation (e.g., an unsafe function call exists at the predicted location).
- **Behavior-backed evidence:** reproduces an expected program path or output condition (e.g., the binary processes the input and reaches the predicted function).
- **Logic-backed evidence:** exercises a controlled harness for the vulnerable state transition (e.g., a test input triggers the expected code path without a full crash).
- **Sanitizer-backed evidence:** reproduces a memory-safety failure with relevant runtime diagnostics (e.g., AddressSanitizer reports a buffer overflow at the predicted location).

The distinction matters because different evidence classes support different claims about the analysis. A sanitizer crash is stronger evidence for a memory-safety vulnerability than a source-code predicate alone. The framework records the accepted evidence class rather than mapping every passing script to the same notion of "confirmed."

3.4. Evidence-Conditioned Feedback

After validation, the system constructs feedback at the run–skill level. This feedback is an operational interface for evolution rather than a standalone causal estimator. Its role is to summarize the relation between task outcome, external confirmation, and observed skill relevance in a form that can drive candidate generation and later gate decisions. In that sense, it is a low-cost approximation to the latent quantity $\Delta(s)$ in Equation 3, but we do not present it as a substitute for full counterfactual attribution.

A skill receives one of four statuses:

Supported ($\hat{\Delta} > 0$):
the task is externally validated and the skill is aligned with the analysis procedure (i.e., the skill's described steps match the agent's actual trajectory). This is positive evidence for the skill, but not proof of causality.

Mismatched ($\hat{\Delta} \approx 0$):
the task may be correct, but the skill is unrelated to the target or method. The model likely succeeded on its own; the skill was redundant.

Contradicted ($\hat{\Delta} < 0$):
the skill appears to drive a false localization, unsupported claim, or failed confirmation. The skill was actively harmful.

Unknown: the available trace cannot distinguish contribution from coincidence.

The key idea is practical rather than metaphysical. Vulnerability-analysis workflows already produce task scores, evidence classes, and auditable records of the selected skill and final answer. We reuse these signals to derive an attribution-oriented label at the cost of one run, whereas counterfactual ablation requires at least two runs and Shapley-style valuation [14] can be exponentially expensive in the number of skill steps. This makes the label useful as a triage signal for evolution even when it remains imperfect as scientific evidence of causal contribution.

However, this approximation has lower precision. A "supported" skill may be supported only by coincidence: the model might have succeeded regardless. A "mismatched" skill might have subtly helped in ways the alignment check cannot detect. We empirically evaluate this precision trade-off in Section 6.

3.5. Candidate-Level Publication Gate

Evolution produces a candidate revision Δs rather than directly overwriting the live skill. Publication is gated by Equation 4. In our prototype, follow-up validation replays the candidate on the same or a closely related task and checks whether the candidate remains consistent with the stored evidence.

The most conservative form of this gate is a *strict improvement* policy: the candidate must achieve a mean validation score that exceeds a baseline score

by a margin. This design is motivated by the concern that successful-but-unsafe or merely coincidental experiences should not be persisted as reusable skills.

However, as formalized in Section 2.3, a replay-only gate has a structural limitation. The follow-up measures $q_{\text{task}}(y_{\Delta s})$ but not $\Delta(\Delta s) = q_{\text{task}}(y_{\Delta s}) - q_{\text{task}}(y_0)$. When the baseline is high (the model can already solve the task), strict improvement is nearly impossible to establish because the candidate's replay score is bounded by the same ceiling. We provide empirical evidence of this limitation in Section 6.3.

3.6. Design Rationale: Live, Candidate, and Share Separation

Live skills, candidate revisions, and shared runtime state are stored in separate directories. This separation is not merely a software-organization choice; it is part of the experimental design. It ensures that candidate skills can be generated, inspected, replayed, and rejected without silently changing the library seen by the next run.

4. Prototype Realization

We realize the framework as a research extension to SkillClaw [3]. The original system already provides request proxying, skill retrieval and injection, session capture, skill sharing, and an evolution service. Our extension does not replace these core services with a new retrieval or revision algorithm. Instead, it adds the elements needed to study the run-to-library transition in a security setting: leakage-controlled benchmark cases, remote blind execution, heterogeneous vulnerability validators, candidate staging, and a structured handoff from finalized runs to validation-gated evolution.

4.1. Analysis Plane and Evidence Plane

The runtime is divided into two planes. The *analysis plane* consists of an LLM agent client on a remote VM that analyzes the target through the SkillClaw proxy. The *evidence plane* is a local service that retains private ground truth, finalizes the run record, executes validators, and sends a closed session/feedback pair to the evolution service. This split preserves the blind-evaluation setting while still allowing rich local post-processing and confirmation.

Client-provided session identifiers are preserved when available, and long sessions are archived as incrementally consumable immutable segments. This

ensures that later interaction can extend a session without rewriting feedback already consumed by the evolver.

4.2. Benchmark Cases

We define benchmark cases in JSON specifications containing: the blind workspace path, the analysis objective (rendered without CVE identifiers or solution-bearing names), private ground truth (expected files, functions, root cause, required evidence terms), and confirmation policies.

The repository-level benchmark library currently contains 27 case specifications, but they are not all used in the same way. We organize them into three evidence tiers:

- **Primary controlled firmware cases:** blind embedded HTTPd command-injection, overflow, and path-traversal tasks analyzed through binary-level reasoning on a remote VM. The main quantitative results in this paper use a frozen 17-case subset from this tier.
- **Open-source confirmation cases:** parser vulnerabilities in giflib, libxml2, tcpdump, libarchive, and Exiv2, where source code, sanitizer runs, and logic-backed validators provide stronger external confirmation. These cases are valuable for validator and gate development, but they are not folded into the frozen 153-run firmware denominator.
- **Supporting diagnostic cases:** earlier development-time or mechanism-isolation cases, including focused CGI profile ablations and decoy-heavy family studies, retained to explain scoring drift, skill structure, and failure modes.

This separation matters methodologically. Firmware cases emphasize blind target localization under binary-only reasoning, whereas the open-source cases emphasize richer confirmation and validator design. Mixing them into a single denominator would blur the distinction between skill effect and evidence availability.

4.3. Scoring

Each case defines a scoring rubric with four check dimensions:

- **File localization:** does the agent identify the correct vulnerable file?

- **Function localization:** does the agent identify the correct vulnerable function?
- **Root cause:** does the agent describe the root cause with sufficient technical detail?
- **Evidence:** does the agent cite concrete evidence (function names, string references, code patterns)?

Each dimension is scored against private case metadata and aggregated onto a 0–10 scale through a weighted rubric. The rubric is deliberately coarse: it is intended to compare blind runs at the task level, not to certify fine-grained semantic equivalence between two explanations.

4.4. Evolution Service

The evolution service consumes the feedback bundle and generates candidate skill revisions using an LLM-based workflow engine. Candidates are content-deduplicated by SHA-256 hash before entering the gate queue. The gate runs follow-up validation and enforces the publication policy defined in Section 3.5.

4.5. LLM Configuration

All controlled experiments reported as primary quantitative evidence use the same backend, `deepseek-v4-flash`, accessed through the local CC-Switch proxy. This does not remove output stochasticity, but it avoids cross-model capability differences within the main comparisons. Earlier development-time diagnostic runs span older runtime snapshots and are used only as supporting evidence for protocol repair, not as the main basis for the paper's claims.

5. Experimental Methodology

5.1. Research Questions

RQ1. Under leakage-controlled blind execution, does skill strength measurably change true-hit rate and decoy-hit rate?

RQ2. When generic skills still fail, can family-specific distinguishing skills recover the target function, and under what boundary conditions?

RQ3. After the loop is operational, what does the publication gate actually guarantee, and what ambiguity remains between genuine improvement and harmless redundancy?

Table 1: Families represented in the controlled benchmark.

Family / device line	Cases	Main challenge
Tenda F1202	2	command injection and credential-related overflow
Tenda F453	4	multiple HTTP handlers with strong `formexeCommand` decoys
Tenda F456	1	command injection near the same Tenda sink cluster
Belkin F9K1122	4 + 2 near-pair cases	family-internal handler confusion around `formWISP5G` and settings
Tenda FH451	5	dense handler clusters with overlapping token patterns
i12	1	path traversal versus adjacent access-control handlers

5.2. Benchmark Cases

The primary benchmark is a frozen set of 17 embedded firmware cases drawn from HTTP handler and CGI vulnerabilities. The controlled studies are run through blind binary-level reasoning on a remote VM, with neither CVE identifiers nor solution-bearing artifact names exposed to the agent. The cases span command-injection, buffer-overflow, and path-traversal localization tasks across several firmware families.

Two additional earlier studies are retained as supporting diagnostics rather than merged into the main benchmark: a 16-run firmware CGI profile ablation over `firmware2-wireless` and `firmware2-login`, and a 16-run Tenda F453 blind diagnostic study that exposed score/feedback misalignment. These runs remain useful for mechanism analysis, but they were produced under earlier development-time protocols and are therefore not treated as the primary evidence base.

5.3. Conditions and Controls

The main evaluation uses three skill conditions:

- **No-skill:** no skill is injected.
- **Weak-skill:** a generic, non-answer-bearing procedural skill is injected (e.g., broad CGI or ELF triage guidance).
- **Oracle-skill:** a case-family-specific skill is injected, still audited to remove direct answer leakage.

We use two controlled protocols:

- **Clean rerun protocol:** 14 cases $\times$ 3 conditions $\times$ 2 rounds = 84 runs. This protocol was introduced to verify that earlier near-perfect oracle performance was caused by answer-bearing skill content.

- **Frozen necessity protocol:** 17 cases × 3 conditions × 3 rounds = 153 runs. This is the main quantitative study reported in the paper.

For the frozen protocol, two F9K1122 near-pair handlers (`formSetSystemSettings` and `formWlanSetup`) are reported separately rather than folded into the per-function denominator, because generic oracle wording cannot reliably distinguish them under token-level blind reasoning. This prevents the aggregate statistics from overstating or understating skill effect on separable targets.

5.4. Complementary Targeted Profile Ablations

We retain four complementary studies for mechanism diagnosis:

- a 16-run firmware CGI profile ablation comparing *none*, *wrong*, *relevant*, and *seed* skills on `firmware2-wireless` and `firmware2-login`;
- a 16-run F453 blind diagnostic study that revealed repeated drift from `formWriteFacMac` to the decoy handler `formexeCommand`;
- a 15-run Belkin F9K1122 family-specific distinguishing study;
- a 15-run Tenda FH451 family-specific distinguishing study.

These studies are not merged into the frozen aggregate. They answer more focused questions about skill structure, decoy attraction, and family-level separability.

5.5. Measurements

We report:

- **Task score:** a 0–10 score decomposed into file, function, root-cause, and evidence sub-scores.
- **Outcome class: YES** (true target hit), **DECOY** (landed on a dangerous but wrong target), or **NO** (miss).
- **Feedback metadata:** including target-aware flags such as `function_identity_miss` when the run is close but lands on the wrong handler.
- **Gate outcome:** published, rejected, or pending, together with the gate mode and baseline relation when available.

5.6. Evidence Hierarchy and Reporting Policy

Not all experimental artifacts produced during system development are equally suitable as paper evidence. We therefore report results under an explicit hierarchy:

- **Primary quantitative evidence:** the 84-run clean rerun, the 153-run frozen necessity study, and the targeted F9K1122/FH451 family-specific distinguishing experiments.
- **Supporting diagnostic evidence:** earlier firmware2 CGI profile ablations and the F453 blind diagnostic study, which are used to interpret failure modes, decoy attraction, and scoring/gate behavior.
- **Retired or non-primary evidence:** development snapshots later found to include answer-bearing skill content, contaminated prompts, or benchmark definitions that were not yet protocol-clean.

This policy is important for two reasons. First, it prevents earlier development-time wins from being mistaken for stable benchmark results. Second, it keeps negative findings visible: studies that exposed leakage, misscoring, or invalid case composition are not deleted, but they are no longer allowed to dominate the paper's main quantitative claims.

5.7. Statistical Analysis

Our analysis is primarily descriptive. The clean rerun protocol provides $N = 2$ rounds per case-condition and the frozen protocol provides $N = 3$. This is enough to expose large directional effects and failure modes, but not enough for strong per-case significance claims. We therefore emphasize aggregate rates, case-level breakdowns, and protocol transparency rather than inferential statistics.

6. Results

6.1. RQ1: Clean Controlled Studies Show Real but Incomplete Skill Effects

Table 2 summarizes the two main controlled protocols.

Finding 1: protocol cleanup was necessary. The clean-rerun study was introduced because earlier oracle runs had produced near-perfect performance under development-time prompts. After removing answer-bearing skill content, oracle accuracy fell to 35.7%. This confirms that the older

Table 2: Leakage-controlled three-condition results.

Protocol	Condition	n	YES	DECOY	NO	Mean score
Clean rerun (84 runs)	no-skill	28	1 (3.6%)	9	18	4.38
Clean rerun (84 runs)	weak-skill	28	4 (14.3%)	12	12	4.34
Clean rerun (84 runs)	oracle-skill	28	10 (35.7%)	1	17	6.32
Frozen necessity (153 runs)	no-skill	45	3 (6.7%)	28	14	3.92
Frozen necessity (153 runs)	weak-skill	45	5 (11.1%)	24	16	3.99
Frozen necessity (153 runs)	oracle-skill	45	21 (46.7%)	2	22	6.20

Frozen-necessity denominator excludes two F9K1122 near-pair handlers that are reported separately under the protocol.

oracle numbers were not acceptable as scientific evidence. The current paper therefore treats the clean and frozen studies, rather than earlier development snapshots, as the main quantitative basis.

Finding 2: skill changes target selection, not just total score. Under the frozen protocol, oracle-skill reaches 46.7% true hits, versus 11.1% for weak-skill and 6.7% for no-skill. More importantly, oracle-skill reduces decoy hits from 28 and 24 down to 2. In other words, the strongest observable effect is not only a score increase, but a sharp reduction in being pulled toward the wrong dangerous handler.

Finding 3: the effect is real but not yet strict necessity. Oracle-skill still leaves 22/45 runs in the frozen denominator as NO. Several cases remain resistant even under oracle guidance, including `f1202-credential-overflow`, `f453-qossetting-overflow`, and `fh451-WrlclientSet-overflow`. The correct claim at this stage is therefore not "the skill is always necessary," but rather "skill strength materially shifts the probability of landing on the true target."

Finding 4: some families are already close to solvable under the current oracle formulation. For example, the frozen study shows `f1202-cmdinject` moving from 1/3 true hits under no-skill and weak-skill to 3/3 under oracle-skill; `f453-overflow` from $0/3, 0/3$ to 3/3; and `f456-cmdinject` from $1/3, 1/3$ to 3/3. These are not universal wins, but they demonstrate that the current skill format can already create strong target-selection differences on some families.

Table 3: Supporting diagnostic studies on skill structure and family-level specificity.

Study	Conditions	Result
Firmware2-wireless	none / wrong / relevant / seed	2.67 / 3.00 / 3.00 / **4.00**
Firmware2-login	none / wrong / relevant / seed	4.33 / 4.50 / **5.00** / **5.00**
Belkin F9K1122	generic oracle / distinguishing oracle	20.0% → **66.7%**
Tenda FH451	generic oracle / distinguishing oracle	40.0% → 26.7%

6.2. RQ2: Skill Specificity Helps Only When the Family Is Separable

We next ask why oracle still fails on many frozen cases. The answer from the supporting studies is that "more specific" is not universally better; it depends on whether the firmware family exposes stable intra-family signals.

Firmware2 profile ablations: structure matters. The earlier firmware2 CGI ablation does not directly measure necessity, but it shows that a branch-first seed skill can outperform a broader descriptive live skill on `firmware2-wireless` while tying it on `firmware2-login`. Topical relevance alone is therefore not enough; the internal organization of the skill can change how the model traverses a CGI dispatcher.

F9K1122: specificity can rescue a family. In the frozen study, generic oracle still leaves the family vulnerable to a `formWISP5G` attractor. The distinguishing F9K1122 skill adds family-level cues such as adjacent symbol and parameter-pattern distinctions, raising the correct rate from 20.0% to 66.7%. This is the clearest positive example in the current repository that a more specific skill can materially recover target localization.

FH451: specificity can also fail. Applying the same design idea to FH451 lowers the family-level result from 40.0% to 26.7%. The lesson is not that distinguishing skills are wrong, but that this style of specificity only helps when the family actually exposes usable discriminative cues. FH451 appears to have denser handler overlap and weaker token separation than F9K1122.

F453: wrong-target high scores motivated target-aware feedback. The earlier 16-run F453 diagnostic study produced only 2 exact hits among 15 valid runs, while 10 runs drifted to the wrong but dangerous handler `formexeCommand`. Many of those wrong-path runs still scored 8.0 because they hit the correct binary and sink family. This is exactly the engineering failure mode that later motivated target-aware metadata such as `function_identity_miss` in the feedback and gate pipeline.

Table 4: Gate decision summary across all experiments.

Metric	Count
Total gate decisions	94
Published	12
Rejected	82
Published under earlier score-only gate logging	10
Published under baseline-aware non-inferiority logging	2

6.3. RQ3: Replay-Only Gate Has a Structural Limitation

We analyze 94 gate decisions collected across all experiments (Table 4).

The archived decision history spans two gate regimes. Earlier decisions record only the candidate-side validation score and final decision. Later decisions additionally record a baseline mean, gate mode, and effective threshold. Among the latter, the two published examples were accepted under a non-inferiority criterion, with $(0.7, 0.4)$ and $(0.6, 0.6)$ for (candidate mean, baseline mean), respectively.

Operationally, the loop is now closed. The repository already contains real candidate → rerun → gate → publish examples, and the gate-settle audit further repaired 21 stuck pending jobs down to 0, set the default `publish_mode` to `validated`, and added unit coverage for the settle logic. This matters because the current question is no longer whether the system can move a candidate through the pipeline; it can. The question is what that publication actually proves.

Root cause analysis. The key issue is not simply that many candidates are poor. Rather, the gate often lacks a defensible basis for deciding whether a candidate skill was *necessary*. In a replay-only setting, a candidate can reproduce the same validated answer as the baseline while providing no clear evidence that it changed the outcome. Under a strict-improvement interpretation, such a candidate is rejected as redundant; under a non-inferiority interpretation, the same candidate may be retained as harmless but potentially useful. The decision therefore depends as much on the gate's philosophy as on the candidate's intrinsic quality.

This is the structural limitation formalized in Section 2.3. The gate's FollowupPass function computes $q_{\text{task}}(y_{\Delta s})$ but only imperfectly captures $q_{\text{task}}(y_0)$. When the model can already solve the task without the skill, a replay-only gate struggles to separate three cases: genuine improvement,

Table 5: Summary of findings mapped to hypotheses.

Hypothesis	Finding	Status
H1: Clean skill effect	Under clean and frozen protocols, oracle-skill sharply outperforms weak/no skill and suppresses decoys, but many cases still remain unsolved	Supported
H2: Specificity boundary	F9K1122 benefits strongly from distinguishing skills, whereas FH451 does not	Supported
H3: Gate limitation	The loop can publish candidates, but replay-only evidence still cannot cleanly separate improvement from redundancy	Supported

harmless redundancy, and spurious success. This ambiguity is exactly why publication remains the hardest part of the loop.

Interpretation. Recent work on skill misevolution [11] proposes SafeEvolve, a wrapper that repairs unsafe content and governs subsequent reuse. Our study highlights a closely related but distinct problem: even when content appears safe, the system may still be unable to decide whether the skill deserves to persist. In other words, safety and usefulness are separable questions, and a realistic evolution loop must reason about both.

6.4. Summary of Findings

7. Discussion

7.1. The Evidence–Attribution Gap

Our results reveal what we call the *evidence–attribution gap*: executable domain evidence can verify *what* the agent found (the vulnerability is real, the localization is correct), but it cannot directly verify *why* the agent found it (was the skill causally responsible?).

This gap has two dimensions:

1. **Positive evidence is not causal evidence.** A run that passes source-backed validation and has a "supported" feedback status may have succeeded because of the skill, because of the model's prior knowledge, or because of both. The four-level feedback reduces this ambiguity by checking skill–trajectory alignment, but it cannot eliminate it.

2. **Absence of evidence is not evidence of absence.** A "mismatched" skill may have subtly influenced the agent's reasoning in ways that the alignment check cannot detect. For example, a skill might prime the agent to look at certain binary sections, indirectly leading to the correct answer even if the skill's specific steps were not followed.

Bridging this gap requires either counterfactual ablation (expensive but precise) or a probabilistic attribution model that estimates $P(\text{success} \mid \text{skill}) - P(\text{success} \mid \text{no skill})$ from observed correlations (cheaper but requires more data). We leave the latter as future work.

7.2. Implications for Skill Library Design

Our finding that the same skill can be helpful, neutral, or harmful depending on the case has a direct implication: *skill libraries need case-sensitive selection and case-sensitive evaluation.* A globally plausible skill may still degrade a specific analysis if its procedural biases do not fit the target. This is consistent with prior observations that skill-induced failures are strongly case-specific [10].

This suggests that retrieval mechanisms should consider not only textual skill–task similarity, but also uncertainty about the model's unaided capability. If the baseline is already strong, injection may add noise, encourage overcommitment to the wrong pattern, or simply consume prompt budget without benefit.

7.3. Toward a Counterfactual Gate

The structural limitation of the replay-only gate (Section 6.3) suggests two directions for improvement:

Scheme A: Counterfactual ablation gate. For each candidate, run the task both with and without the candidate skill, and require strict improvement ($\bar{y}_{WS} > \bar{y}_{NS}$). This directly measures $\Delta(\Delta s)$ but doubles the validation cost and requires statistical power (multiple rounds).

Scheme B: Non-inferiority gate. Instead of requiring strict improvement, require that the candidate is *not worse* than the baseline by more than a margin ϵ: $\bar{y}_{WS} \geq \bar{y}_{NS} - \epsilon$. This relaxes the gate to allow redundant (but not harmful) candidates, at the cost of potentially publishing skills that do not help.

A combined approach could use Scheme B for initial publication (allowing the skill to enter the live library) and Scheme A for retention decisions

Table 6: Comparison with concurrent work on agent skills.

	Attribution		Publication		
Approach	Method	Cost	Method	Safety	Domain
Ours	4-level evidence	1 run	Replay gate	Conservative	Vuln. analysis
SkillShapley [14]	Shapley value	$O(2^n)$	N/A	N/A	General
Skill Harmful [10]	Differential	2 runs	N/A	N/A	General
Misevolution [11]	N/A	N/A	SafeEvolve	Repair-based	General

(removing skills that do not demonstrate improvement over a longer horizon). More broadly, this suggests that publication and retention may need to be decoupled: the system can admit a candidate as provisionally safe while still demanding stronger evidence before treating it as established expertise.

7.4. Comparison with Concurrent Work

Table 6 compares our approach with concurrent work on agent skills.

Our approach is distinctive in its use of executable domain evidence as a post-run attribution signal. This gives it a cost advantage over explicit ablation, but also a narrower claim: we obtain an auditable approximation, not a full causal decomposition.

7.5. Limitations of Static Analysis

The firmware cases in our A/B study are analyzed through binary-level static reasoning without a live runtime environment. Consequently, the available evidence is weaker than sanitizer-backed confirmation in the open-source cases. This likely lowers the precision of the feedback signal and should be interpreted as a limitation of the present study, not as a property of the framework in general.

8. Related Work

8.1. Skill Induction, Libraries, and Evolution

Voyager [1], ExpeL [2], Reflexion [16], and MemGPT [17] establish the broader agent-learning context in which trajectories, memories, and reusable instructions are accumulated across tasks. SkillClaw [3] makes this logic explicit at the level of a shared natural-language skill library, while Ctx2Skill [4] and OPID [5] study, respectively, context-to-skill induction and trajectory-derived skill distillation. These works show that skill formation and reuse

are already central to LLM agent research. Our paper does not compete by proposing yet another generic skill generator. Instead, it focuses on a narrower systems question: in a vulnerability-analysis deployment, how should a completed run be transformed—or refused transformation—into a live reusable skill?

8.2. Environment- and Verifier-Grounded Skill Refinement

Recent work increasingly grounds skill refinement in environmental or verifier feedback rather than in free-form self-reflection alone. SPARK [6] emphasizes *evidence over plans*, preferring environment-grounded revisions to purely narrative improvement. Trace2Skill [8] distills trajectory-local lessons into longer-term skills using verifier feedback, and VeriSkill [9] studies self-evolution specifically for program verification skills. SkillEvolBench [7] further turns episodic-to-procedural evolution into an explicit benchmark problem.

Our work is aligned with this trend, but differs in scope and target. We do not study general verifier-backed skill improvement in the abstract. We study the security-specific transition from *blind remote vulnerability-analysis runs* to *candidate skills that may alter a live library*. In this setting, the central issue is not only whether verifier signals exist, but how they participate in archival, candidate staging, and publication control.

8.3. Skill Failure, Attribution, and Governance

Skill-induced failures [10] demonstrate that apparently relevant skills can actively harm agent performance by steering execution onto the wrong procedure. Skill misevolution [11] shows that self-improving loops can preserve unsafe behavior as persistent skills. Adversarial skill publishing [18] further exposes how untrusted skills can manipulate downstream behavior.

These works motivate our emphasis on post-run governance. In our setting, retrieval and generation are only the front half of the problem. The back half is deciding whether a candidate skill deserves admission into the live library. Our gate analysis complements prior work by showing that even when candidate generation is available, publication remains difficult because replay-only evidence is often insufficient to separate improvement from redundancy.

8.4. Skill Attribution and Valuation

SkillShapley [14] formulates skill-step attribution through Shapley-value estimation [15], yielding a principled but costly counterfactual approach.

Differential no-skill / with-skill comparison [10] provides a cheaper alternative at the run level. Our feedback interface belongs to this family of post-run assessment methods, but with a deliberately narrower claim: it is a triage-oriented operational signal, not a full causal estimator. Its purpose is to support candidate generation and publication decisions in a security workflow where full ablations are expensive and often delayed.

8.5. Security Agents, Playbooks, and Benchmarks

On the security side, PentestGPT [19] and subsequent studies on LLM-based vulnerability analysis [20, 21] focus on what models can discover or explain. More recent work such as EvoHunt [12] begins to evolve reusable playbooks for agentic security auditing, while SEC-bench Pro [13] raises the bar on realistic long-horizon software-security evaluation. Our work is adjacent to both directions. Like EvoHunt, we care about reusable procedural knowledge in security tasks; like SEC-bench Pro, we care about realistic task construction and leakage control. The difference is that we center the run-to-library transition itself: blind execution, external confirmation, candidate staging, and live-skill admission are the primary objects of study.

9. Threats to Validity

Sample size.. The main controlled studies are larger than the early six-case A/B stage, but per-case repetition is still limited: the clean rerun uses $N = 2$ rounds per condition and the frozen protocol uses $N = 3$. This is enough for large directional comparisons, but not enough for strong per-case significance claims.

Model variance.. LLM outputs remain stochastic. The value of the frozen protocol is that it lets us observe large aggregate shifts despite that noise, but borderline cases can still flip across rounds. Future work should either increase N or explicitly combine repeated runs with counterfactual comparisons.

Single model.. The primary quantitative results use one model family (`deepseek-v4-flash`). Skill effects may differ across models, as different models have different priors and instruction-following behavior. Cross-model validation is left as future work.

Static analysis limitation.. The firmware cases in the main studies are analyzed through binary-level static reasoning without a live runtime environment. Consequently, the available evidence is weaker than sanitizer-backed confirmation in open-source cases. This likely lowers the precision of the feedback signal and should be interpreted as a limitation of the present study, not as a property of the framework in general.

Attribution precision.. The feedback labels and target-aware miss tags are still heuristic approximations, not causal estimators. A run that lands on the correct function under oracle guidance may still have been recoverable without that skill, while a run marked as a miss may still have been partially helped by skill-induced attention shifts. Rigorous causal validation still requires counterfactual ablation.

Gate evaluation.. The 94 gate decisions were produced under multiple development-time gate regimes. Earlier decisions preserve only candidate-side validation outcomes, whereas later decisions include baseline-aware metadata and non-inferiority parameters. This historical mixture is useful for retrospective diagnosis but imperfect for a clean scientific comparison. A future study should evaluate gate variants from fixed repository snapshots under a single benchmark protocol.

Answer leakage.. Known-CVE benchmarks can leak through identifiers, artifact names, comments, or pre-existing PoCs. While our blind workspaces are generated automatically and audited to exclude solution-bearing material, we cannot guarantee zero leakage, especially for well-known CVEs. The clean rerun study itself is evidence that this threat is real rather than hypothetical.

Oracle validity.. Case validators encode trusted expectations and may themselves be incomplete. A validator that passes does not guarantee the vulnerability is fully understood; a validator that fails does not guarantee the analysis is wrong. We therefore report validator provenance and keep some cases as supporting diagnostics rather than promoting them to the main evidence set.

Benchmark curation and invalidated cases.. Part of the experimental contribution in this project is procedural rather than purely numerical: several earlier case configurations had to be reclassified, split out, or removed from the main denominator after we found leakage, duplicate-family ambiguity, or misleading score inflation. This means the benchmark is not a static artifact

handed down in advance, but an evolving research object that had to be cleaned before it could support defensible claims. We mitigate this risk by preserving supporting records, explicitly separating primary and diagnostic evidence, and reporting when a case remains useful for engineering diagnosis but not for the main aggregate.

10. Conclusion

We studied the transition from blind vulnerability-analysis runs to live reusable skills. The main claim of the paper is not that autonomous skill improvement is solved. Rather, we show that this transition can be made explicit enough to study: blind remote execution, local external confirmation, run archival, candidate generation, and publication control can be connected into a single operational loop.

The empirical picture is now clearer than in the earlier development stage. After removing answer-bearing skill content, clean reruns show that oracle performance remains substantially above weak/no controls rather than collapsing to zero. The larger frozen protocol strengthens that result: oracle-skill reaches 46.7% true hits versus 11.1% and 6.7%, while reducing decoy hits from 28/45 and 24/45 to 2/45. At the same time, the effect is not uniform. Family-specific distinguishing skills help F9K1122 dramatically but harm FH451, and several frozen cases still remain unsolved even under oracle guidance.

We also show that publication control is the real bottleneck of the current loop. The system can already produce candidates, rerun them, and publish some of them into the live library. The harder problem is not candidate generation but deciding whether a validated run contains knowledge that truly deserves to persist. Replay-only gates can enforce safety and non-degradation, but they still do not by themselves prove causal skill contribution.

This observation is relevant beyond vulnerability analysis. If the run-to-library transition is difficult even in a domain with executable external confirmation, it is likely harder in agent settings where such confirmation is weak or absent. Our current framework suggests a pragmatic stance: external evidence can discipline skill evolution, but it does not remove the need for explicit counterfactual comparison, publication policy, and retention policy.

Future work should proceed along four directions: (1) extend the frozen protocol to more case families and more rounds; (2) add counterfactual ablation or retention-stage gates that better separate improvement from

redundancy; (3) broaden the use of runtime-backed evidence beyond current firmware cases; and (4) compare server-side inline injection with catalog-style selection under the same controlled benchmark.

Acknowledgments

Acknowledgments are omitted in the current working draft.

References

[1] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. An, Voyager: An open-ended embodied agent with large language models (2023). arXiv:2305.16291.

[2] A. Zhao, D. Huang, M. Yan, L. Duan, Y. Shao, S. Yuan, M. Yi, Y. Liu, Expel: Llm agents are experiential learners (2023). arXiv:2308.10144.

[3] Z. Ma, S. Yang, Y. Ji, X. Wang, Y. Wang, Y. Hu, T. Huang, X. Chu, Skillclaw: Let skills evolve collectively with agentic evolver, arXiv preprint arXiv:2604.08377 (2026).

[4] S. Si, H. Zhao, Y. Lei, Q. Wang, D. Chen, Z. Wang, Z. Wang, K. Luo, Z. Wang, G. Chen, F. Qi, M. Zhang, M. Sun, From context to skills: Can language models learn from context skillfully? (2026). arXiv:2604.27660.

[5] S. Yang, J. Wu, Z. Lu, Y. Shen, F. Zhang, L. Feng, S. Zhang, H. Luo, Z. Lian, Z. Wen, J. Tao, Opid: On-policy skill distillation for agentic reinforcement learning (2026). arXiv:2606.26790.

[6] N. T. Josyula, H. Bansal, A. Dancu, A. Cebuc, D. Mishra, D. Jaucanu, M.-C. Popescu, Y. Abu-Mostafa, A. Dragan, D. Iliescu, Spark: Environment-grounded skill refinement with evidence over plans (2026). arXiv:2605.09192.

[7] J. Wei, Y. He, K. Zhao, Z. Sun, Y. Wen, J. Xu, Y. Wang, Y. Tsvetkov, Y. Choi, Skillevolbench: Benchmarking the evolution from episodic experience to procedural skills (2026). arXiv:2605.07358.

[8] W. Zhao, B. Zhang, J. Su, Y. Wang, Y. Gao, Z. Hu, H. Chen, Trace2skill: Distilling trajectory-local lessons into long-term skills with verifier feedback (2026). arXiv:2603.25158.

[9] Z. Sun, Y. Shi, F. Long, S. Wang, Y. Liu, Veriskill: A self-evolution framework for program verification skills (2026). arXiv:2606.01139.

[10] Anonymous, Agent skills can be harmful: An empirical study of skill-induced failures in llm agents (2026). arXiv:2608.11888.

[11] Anonymous, Practice makes unsafe: Skill misevolution in self-improving llm agents (2026). arXiv:2608.12851.

[12] B. Liu, P. Gurevich, L. Hext, C. Rister, C. Castillo, C.-C. Tsai, H. Sun, Transferable self-evolving playbooks for agentic security auditing (2026). arXiv:2606.16420.

[13] H. Lee, J. Liu, D. Kim, W. Xia, Z. Zhang, C. S. Xia, L. Zhang, Sec-bench pro: Can language models solve long-horizon software security tasks? (2026). arXiv:2604.19195.

[14] Anonymous, Skillshapley: Boundary-adaptive shapley valuation for skill step attribution in llm agents (2026). arXiv:2608.13173.

[15] L. S. Shapley, A value for n-person games, contributions to the Theory of Games, Vol. 2 (1953).

[16] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning (2023). arXiv:2303.11366.

[17] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, J. E. Gonzalez, Memgpt: Towards llms as operating systems (2023). arXiv:2310.08560.

[18] Anonymous, Convergent detour hijacking: Task-preserving resource amplification in skill-based llm agents (2026). arXiv:2608.12273.

[19] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Desmet, P. Liu, R. Pang, Y. Su, T. Zhang, S. Picek, C. Tripp, Pentestgpt: An llm-empowered automatic penetration testing tool (2023). arXiv:2308.06782.

[20] Steenhoek, Benjamin, Le, Viet, Huang, Wei, Zhang, Hua, Jiang, Lingming, Ray, Baishakhi, Software vulnerability detection using large language models, comprehensive empirical study on LLMs for vulnerability detection (2024).

[21] Anonymous, Automated security learning and reasoning: A survey on llm-based vulnerability detection, survey on LLM-based vulnerability detection approaches (2024).